Software Engineering I (02161)

Exam Project

Assoc. Prof. Hubert Baumeister

DTU Compute
Technical University of Denmark

Exam project

- Problem:
  - Design and implement a stand-alone project planning and time recording system
  - **UI required**, but not a *graphical UI*
  - storage of data in database or in a file is *not required*
- Schedule
  - Week 6 + 7: **Report 1**: Requirements and Design
  - Week 8: **Peer Review** Report 1
  - Week 9 – 12: **Implementation** and **Report 2**
  - Week 13: 15min **Demo**

→ Detailed dates and problem description on DTU Learn (Peergrade and Assignment module)

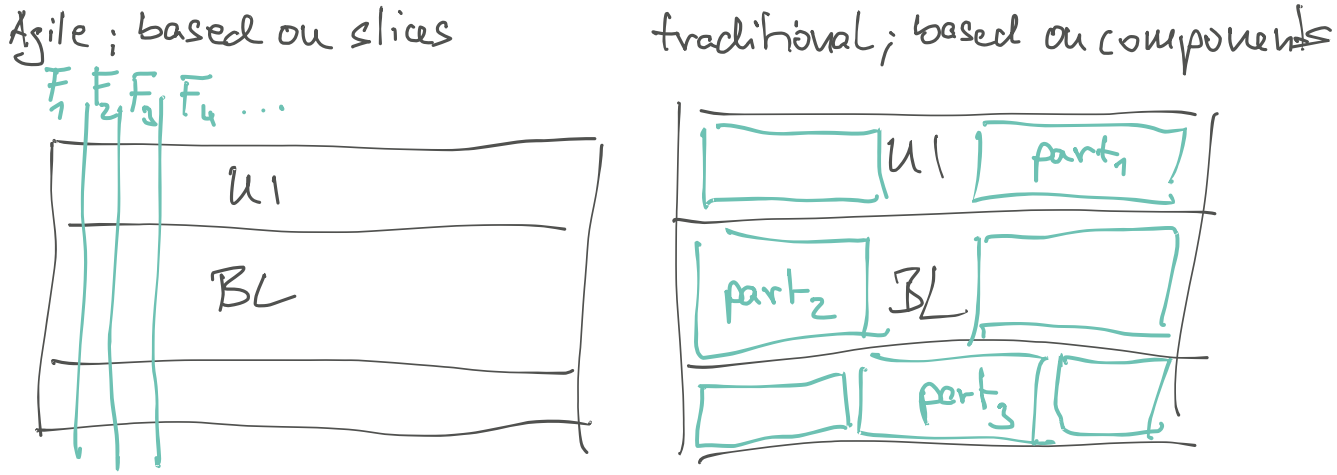
Mark you contributions

- Each section, diagram, class, method, etc. needs to name the author who made the section, diagram, etc.
  - DTU requirement for group submissions
- **You can talk with other groups (or previous students that have taken the course) on the assignment, but *it is not allowed to copy from others parts of the report or the program*.**
  - *Any copying of text without naming the sources is viewed as cheating*
- In report 2 you can describe how you worked together
- Self evaluation, a real number between 0 and 2
  - 0 means no contribution
  - 1 normal contribution
  - 2 twice the normal contribution

	student-1	student-2	student-3	student-4	student-5
Bidrag	1.0	1.5	0.5	0.0	0.5

Recommendations

- Development in slices of **UI** and **BL together**



  - → In two weeks: exercise on developing in slices
  - YouTube video (`https://youtu.be/1vi25KLyEHg`)
- Mob programming (driver, navigator, mob)
  - → Next week: exercise on Mob programming
  - Use for report 1, peer review, report 2, and your implementation
  - Important:
    - change driver after 5-10 minutes
    - driver acts as an *intelligent keyboard*
- Pair programming
  - Possibly later in the development of the application

Grading

- Grading
  - The project will be graded as a whole
  - Evaluation criteria
    - Correct use and understanding of the techniques introduced in the course
    - Implementation: good architecture, understandable code and easy to read (e.g. short methods, self documenting method names and variable names, use of abstraction)
    - Rather focus on a subset of the functionality with good code quality than on having everything implemented but with bad code quality
    - Sufficient tests ($\rightarrow$ code coverage) and quality of tests
- Important
  - Lectures continue until and including **week 10**
  - Topics taught in these lectures are **relevant for the exam project**